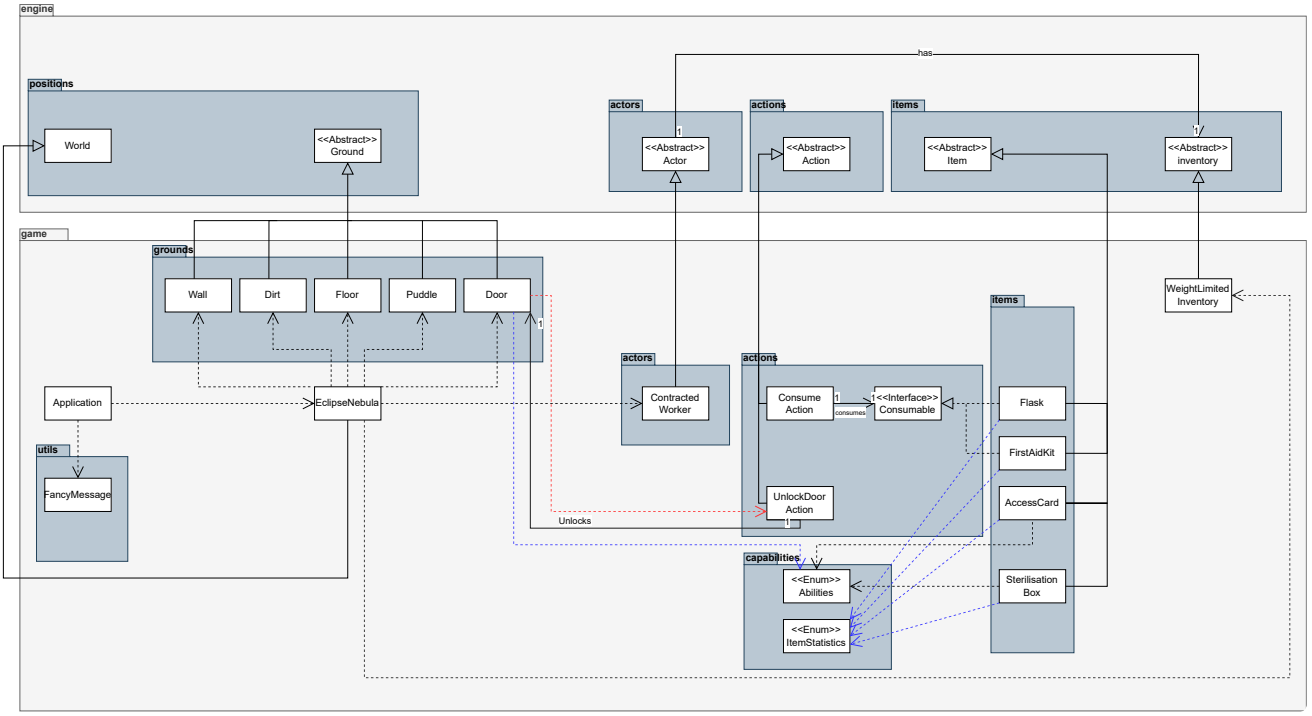
FIT2099 Assignment 1 Design Rationale

REQ1 Basic Items and Ship

REQ1 Setup the basic senario of workers and ship. Every worker spawns with an inventory limited to 50 units of weight and a Flask allow 5 consumptions. The ship is composed of wall `#`, floor `_`, and locked doors `=` . Three unique items can be found on the ship -- AccessCard (unlock doors), FirstAidKit (heal with cooldown), and SterilisationBox (sterlize).



1.1 Inventory Weight Limit

Requirement: The inventory of workers have a weight limit of 50.

Solution: Creates `WeightLimitedInventory` which extends `Inventory`, override `add()` to return `false` when overweighted. Items without `WEIGHT` will tirgger `IllegalArgumentException` -- all Items that added to the `WeightLimitedInventory` must declare a weight.

Pros	Cons
utilized engine mechanise: `PickUpAction` automatically handles the situation where `add()` returning `false` and display the Error message; Rules are encapsulated within its own class	additional subclasses created

OOP principle: `WeightLimitedInventory` is a specific kind of `Inventory` that can be handled as an `Inventory` , which adheres **Liskov Substituion Principle**

1.2 Flask and FirstAidKit -- Consumable Interface (Decision)

Requirements:

- Flask: weights 3, can be consumed 5 times, heal 1 HP each. keep in inventory when depleted.
- FirstAidKit: weights 25, +1 max HP and full heal. 20 turns of cooldown; cooldown only counts when carried.

Alternative A: Separate `Action` classes : `ConsumeFlaskAction`, `ConsumeFirstAidKitAction`, each with their own effect logics.

Alternative B: Create `Consumable` interface that has method `consumedBy()` and `canBeConsumed()`. Both Flask and FirstAidKit implements it. The same kind of `ConsumeAction` handles all `Consumables` , call `consumedBy()` without refering exact class type.

	Pros	Cons
A	Actions are individual; Easy to understand	Action classes will multiply fast. Fails the purpose of the abstraction of `Action`.
B	new consumable items can Implement `Consumable`, and use same `ConsumeAction`, makes it easily extensive	Common attributes cannot be shared at the interface level, each implementing class must declare its own fields; similar logic repeated across classes cannot be extracted upward

Justification: Chose B. Two consumables in REQ1 already, and we have even more in REQ2 (Apple, Cookies, Puddles). It is inefficient to write a corresponding action for every Item. `ConsumeAction` achieves **Polymorphism** through `Consumable.consumedBy()` . New consumables requires only implementation instead of changing code of the `Action`, which adheres to **Open-Closed Principle**. Durability of `Flask` and cooldown of `FirstAidKit` is internal detail maintained by classes their own, which adheres to the **Single Responsibility Principle**.

1.3 Door & SterilisationBox -- Ability Tag (Decision)

Requirements:

- There are locked doors on the ship (`=`); AccessCard can open any doors
- SterilisationBox can sterlize polluted items (poison → heal).

Alternative A: Check specific class type. Door checks `inventory.contains(AccessCard.class)`; Consumables check `inventory.contains(SterilisationBox.class)`.

Alternative B: Items enable Ability tags. AccessCard enables `OPEN_DOOR`; SterilisationBox enables `STERILISE`. Door checks `actor.hasAbility(OPEN_DOOR)`; Consumables check `actor.hasAbility(STERILISE)`.

	Pros	Cons
A	Straight foward without additional mechanism	New door opening tools → must change all checks
B	Item with same ability are abstracted into same tag, new item with the ability only needs to add the tag	Must handle the tags correctly during operations. Bugs in tags tends to be silent and harder for debugging.

Justification: Choose B. In accordance to **Dependency Inversion Principle**, Door should not rely on specific kind of key to unlock it. Same applies to SterilisationBox: Consumables that can be Sterlized checks `hasAbility(STERILISE)` and does not care the specific item providing such ability. Adding new items with similar or same ability does not require change in current class code, this adheres **Open-Closed Principle.**

1.4 Flask consumption behavior

Requirements:

- Flask stays in the backpack after depletion

Approach: Unlike `Apple` and `Cookies` the `consumeBy()` in flask never calls inventory.remove(this).

OOP principle: Flask maintain's it's own behavior during consumption. We dont want `ContractedWorker` or `ConsumeAction` to handle something beyond themselves. Keeping this logic inside adheres to the **Single Responsibility Principle**.

1.5 FirstAidKit cooldown

Requirements:

- Requirement: Cooldown only counts down when carried

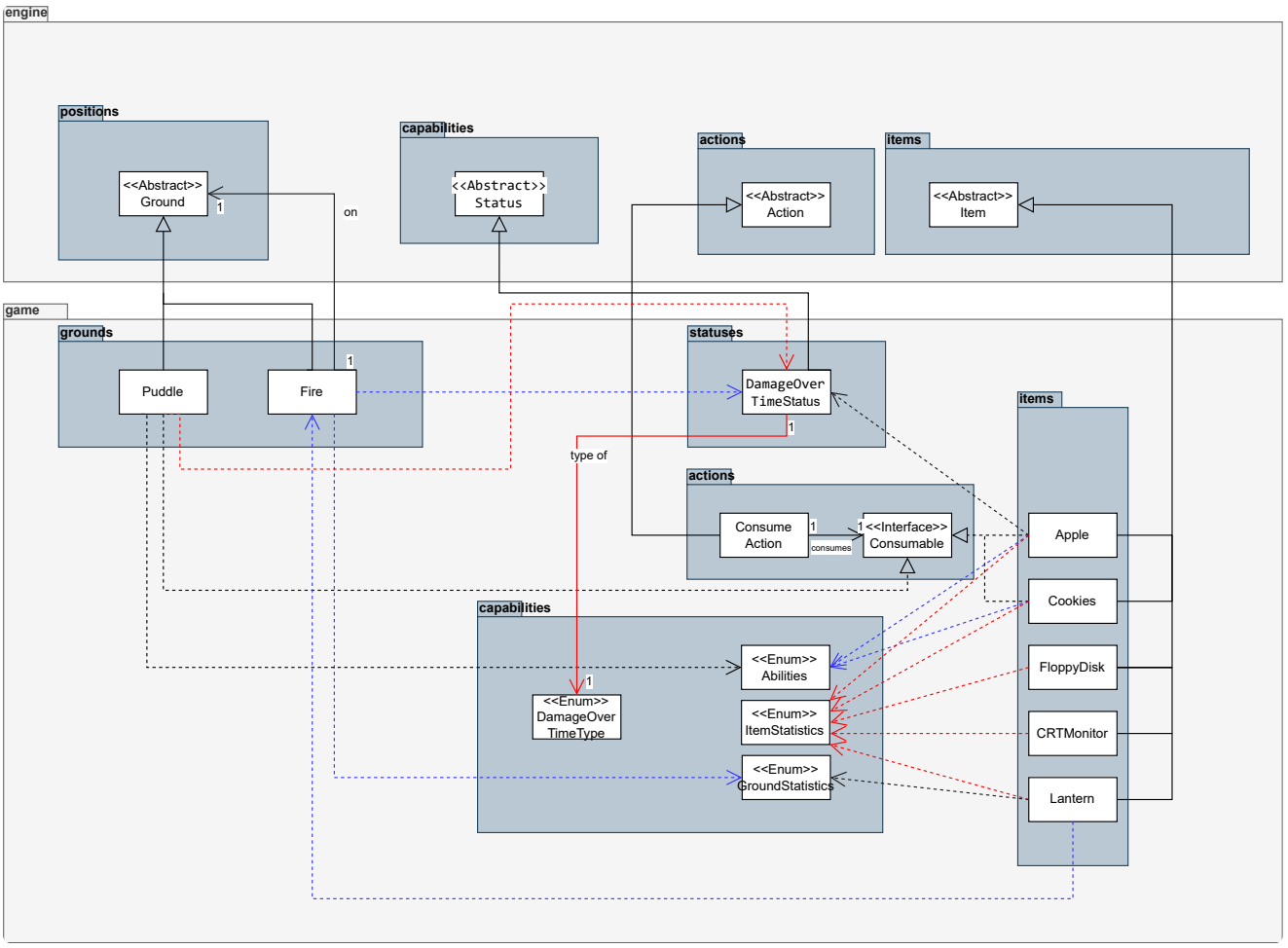
Approach: FirstAidKit overrides `tick(Location,Actor)`, this method is only called when the `Item` is inside an `Actor`'s inventory.

Why: This utilized provided engine mechanism.

REQ2 DoT effect, Environment Interaction and more Items

REQ2 introduces new Items, DoT mechanisms and environment Interactions. Workers can find harzardos consumables such as Apple and Cookies that can be sterilized with

SterilizationBox. Two simple items: FloppyDisk and CRT Monitor with only weight feature.
A Lantern that leaks oil and cause fire on grounds. The puddle can now be drinked directly.



2.1 DamageOverTimeStatus

Requirement:

- Apple: poison player 1 dmg/turn for 5 turns when unsterilized
- Puddles: poison player 1 dmg/turn for 3 turns when unsterilized
- Fire: apply a burn effect 1 dmg/turn for 5 turns when and `Actor` stand on it.

Approach: Create a `DamageOverTimeStatus` and `DamageOverTimeType` enum as constructor arguments.

Reason: The logic inside every DoT effect are all "x damage per x turn" form. Instead of handling them separately, It is clearly more efficient to abstract them into a general status differentiated with an enum. This is an example of **Polymorphism**.

2.2 Puddle -- Utilize Consumable Interface (Decision)

Requirement: Puddle can be drinked directly form ground and cause effects, poison player 1 dmg/turn for 3 turns when unsterilized and heal 1 when sterilized.

Alternative A: Add logic in `Puddles` or `ContractedWorker` to drink on puddle kind of ground.

Alternative B: Utilized and Implement the Consumable interface earlier to handle the drinking of puddle through `ConsumeAction` and maintain the logic in `ConsumedBy()`

	Pros	Cons
A	Item and Puddle is handled separately and avoid interference	Redundancy in logics since the logic is being similar.
B	A single `ConsumeAction` handles all similar kind of action, no matter what exactly it is. Decrease code logic redundancy	There lacks a way to distinguish if a `Consumable` is an `Item` or `Ground`.

Justification: Chose B. `Consumable` interface has already been estabilished in REQ1. This modification is not happening in any of previous code, which adheres to **Open-Closed Principle**. `ConsumeAction` handles all `Consumable` things, no matter what exactly it is, this is a kind of **Polymorphism** and gives better extensibility.

2.3 Apple & Cookies -- Sterilization Ability check

**Requirement:

- Apple: weights 1, poison player 1 dmg/turn for 5 turns when unsterilized, heal 3 hp when sterilized. Removed after consumption.
- Cookies: weights 2, with five pieces inside, each one -1 max hp when unsterilized, heal 1 when sterilized. Removed after all consumed.

Approach: Both Items Implements `Consumale` and check in `consumedBy()` see if the actor consuming it have the ability of Sterilization, choose different path of outcome and use `DamageOverTimeStatus` for DoT effects. The removal logic is achieved same as `Flask` in REQ1 where it depends how `Item.remove()` is handled inside their `consumedBy()`.

Reason and OOP principle:

- `hasAbility()` utlized engine freatures for abstraction.
- The multiple use logic handled within cookie itself, this adheres to the **Single Responsibility Principle**
- Both item does not directly interaction with `SterilizationBox`, only the abstract tags, future items with sterilization ability only needs to opt in through the abstraction layer, which adheres to **Dependency Inversion Principle**

2.4 Lantern

Requirement: Weights 7, with 10 units of oil. 5% chance of leaking 1 unit of oil and creating a `Fire` on the ground each turn.

Approach: The `Lantern` overrides `Item.tick(Location,Actor)` , spawn fire when it leaks and decrease the fuel counter by 1. The fuel and the probability of leaking is written as constructor parameters which allows further customization.

OOP principle: The leak's logic is handled inside lantern, the Item manage its own logic, this adheres to the **Single Responsibility Principle**.

2.5 Fire -- Wrapper

Requirement: A fire exists on ground for 5 turns, apply a stack of buring effect of actor on it each turn, and extinguish after that.

Approach: `Fire` is considered a new subclass of `Ground`. It caches the original ground with `previousGround`, and restore the orginal ground when extinguished.

OOP principle: `Fire` introduces the inflaming of ground as a wrapper around normal `Ground`s, current `Ground` subclasses does not require any change. This adheres the **Open Closed Principle**. Also this is not considered a decision since it is not really practical to maintain two states in each `Ground` subclass, the engine does not support it.

2.6 Floppy Disk & CRT Monitor

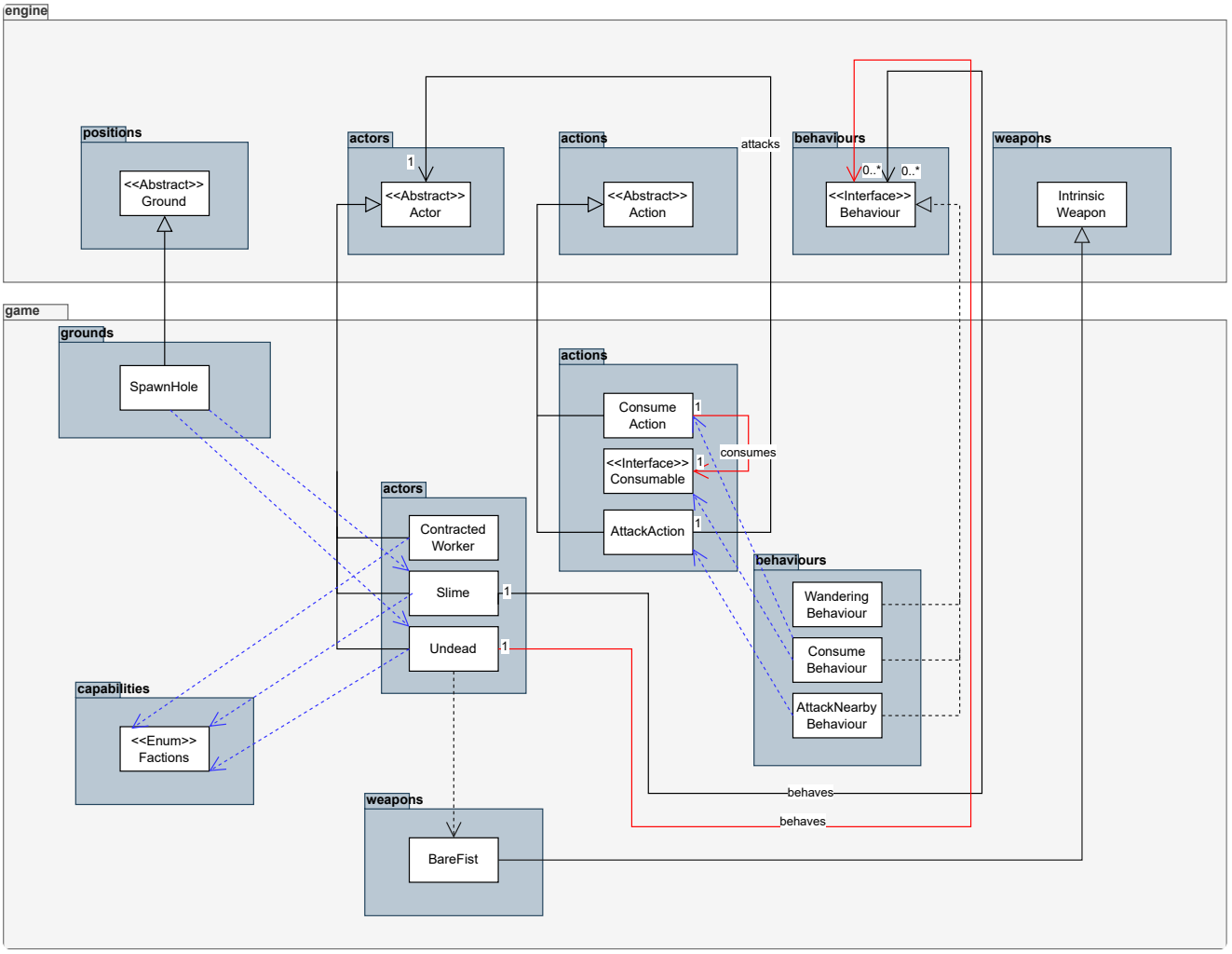
Requirement: `FloppyDisk` weights 1; `CRTMonitor` weights 30. Both Items has no specific use except being scrap that can be carried.

Approach: Both extends `Item`, utilize `GameEntity.addNewStatistic()` with `ItemStatistics.WEIGHT` to register their weight.

Reason: Utilized existing engine feature.

REQ3 Moon Creatures and Behaviour System

REQ3 introduces 2 new type of creatures, undead `X` and slime `ᗝ` , and a hole `o` ground type that spawns one of the two creatures every 20 turn. Both creature wanders when idle. If there is a worker nearby, the undead will attack the worker. The silme will consume any consumables it finds on the current tile.



3.1 Undead, Slime and Behaviour System (Decision)

Requirement: Undead attacks any worker nearby. Slime consumes any consumables on ground. Both creatrues will wander when idle.

General approach: both creatures extends `Actor` classes.

For specific behaviour:
Alternative A: write the logic procedurally inside the `playTurn()` of each creature.

Alternative B:

- use a `TreeMap<Integer, Behaviour>` in every creature with a behaviour to maintain priority between behaviors.
- Create behaviour classes such as `AttackNearbyBehavior` that implements `Behavior` and yield specific `Action` accrodingly.
- For undead, the `AttackNearbyBehavior` will be prioritzed over the `WanderBehavior`.
- For Slime, the `ConsumeBehaviour` will be prioritized over the `WanderBehaviour`.
- `playTurn()` will check the tree map and find the first `Behaviour` that is not yielding `null`.

	Pros	Cons
A	Logic is compact and easy to understand	The `playTurn` will become bloated when new behaviours is required. Similar code is repeating in `Slime` and `Undead`. New behavior requires change in `playTurn()`
B	New behavior only requires adding a new `Behavior` subclass and register it in creature's behaviour tree.	Behaviour logic is distributed across multiple classes; understanding a creature's full turn requires tracing through several files. The priority ordering is implicit in TreeMap keys misconfiguring keys leads to wrong behaviour without compile-time errors.

Justification: Chose **B**. Adding new behaviors does not require changes in existing classes, this adheres to the **Open-Closed Principle(OCP)**. In contrast, **Alternative A** will require change in `playTurn` of the creatures, this is against **OCP**. This also provided the extension point for REQ4 to interact by swapping behavior tree items to actively control behavior.

3.2 ConsumeBehavior

Requirement:

- This is a subsequent requirement due to previous decision in **3.1**.
- Silme will consume any consumable items that worker can consume, gain or suffer the same effect as workers.
-
  - This behavior of slime has been abstracted into `ConsumeBehavior` and reuseable for other creatures in future.

Approach: the `ConsumeBehaviour` utilize engine method `Location.getItemsAs(Consumable.class)` to filter consumables on the ground instead of invoking `instanceof`.

OOP principles: Adding new consumables in the future does not require changes in `ConsumeBehavior` , this adheres to the **Open-Closed Principle**.

3.3 AttackNearbyBehavior and Faction -- Faction Tag over `instanceof` (Decision)

Requirement:

- This is a subsequent requirement due to previous decision in **3.1**.
- Undead have a **Identification Friend or Foe(IFF)** mechanism and attacks only Workers, ignoring Slime and other creatures.
- This behavior of undead has been abstracted into `AttackNearbyBehavior` and reuseable for other creatures in future.

Alternative A: `AttackNearbyBehavior` checks `instanceof ContractedWorker` to identify targets.

Alternative B: Use a `Faction` enum. Workers belong to `Factions.PLAYER`; Undead's `AttackNearbyBehavior` only targets actors with that faction tag.

	Pros	Cons
A	Straightforward; no extra mechanism needed	`AttackNearbyBehavior` depends on a specific class; new player-like actors need to be added to the check
B	Every new actor can assign themselves with corresponding faction for **IFF** purpose.	Must correctly assign factions to all actors

Justification: Chose **B** . The `Faction` enum abstracts "Who is a player" and "Who is monster" from specific `Actor` classes like `ContractedWorker` and `Undead`, the `Behaviors` now behaves hostility according their `Faction` settings, both `Behavior` and `Actor` relies on the `Faction` instead of eachother, this is an application of **Dependency Inversion principle**. Adding a new actor into the hostility system only requires assigning a `Faction` `Ability` for them, this adheres to the **Open-Close principle**.

3.4 SpawnHole

Requirement: A hole `o` spawns an undead `X` or slime `ᗝ` every 20 turns.

Approach: `SpawnHole` extends `Ground`, maintains an internal counter and overrides `Ground.tick()` to achieve cooldown. Spawning new creatures with `Location.addActor()`, only when the hole is not blocked by any `Actor`. The cooldown will still reset even the spawn was blocked.

Reason: This utilized the exisiting `Ground.tick()` from engine.

REQ4 Facility Alarm System

REQ4 (HD) introduces an alarm system on the moon and require one custom trigger and two consequences. Requires the design of a robust, scalable way for the alarm system to interact with the game's existing entities, actors, and map.

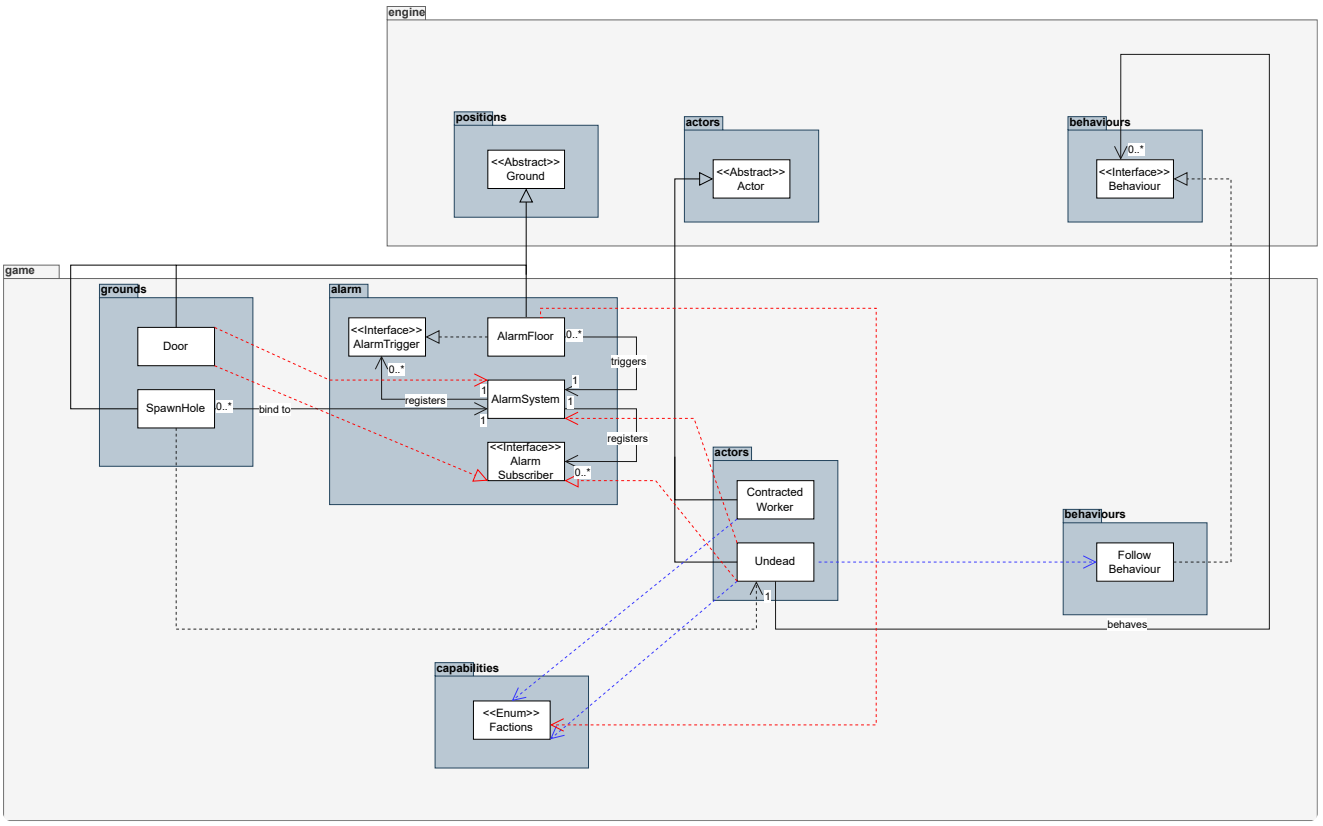
Here is the trigger and consequences I choose.

Trigger:

- A new type of floor or tile that silently triggers the alarm as soon as a worker steps onto it.

Consequences:

- All hostile entities (like the Undead) immediately override their standard wandering behaviour. They must move directly toward the worker, regardless of their proximity or distance on the map.
- All doors are locked tight, and tools, including the AccessCard (目), cannot be used to open them for a certain number of turns.



4.1 AlarmSystem and AlarmSubscriber (Decision)

Requirement: A robust and scalable alarm system that has two consequences when triggered.

Alternative A: The system will directly change the behavior of creatures by sweeping the map, finding all the hostile creatures and doors, change their state accordingly.

Alternative B: An `AlarmSubscriber` interface with method `onAlarmTriggered()`. AlarmSystem maintains a `List<AlarmSubscriber>` and calls `onAlarmTriggered()` on all subscribers when triggered. `Undead`, `Door`, and `SpawnHole` receive `AlarmSystem` via constructor injection and register themselves.

Alternative C: An abstract `AlarmConsequence` class representing the consequences, and Implemented as `LockdownConsequence` and `HuntdownConsequence` for specific alarm consequence logics. The `AlarmSystem` will register them and trigger them during the alarm.

	Pros	Cons
A	No extra interfaces or classes; logic is centralised in one place	AlarmSystem must know the internals of every responder type; new consequences requires modification in AlarmSystem
B	When there is new consequences, implement the `AlarmSubscriber` method on those responding to the alarm.	Alarm response logic is spread across subscriber classes rather than centralised; each responder type must remember to subscribe, a missed registration fails silently. Adding a new consequence type still requires modifying every relevant subscriber class.
C	Consequence logic is centralised in dedicated classes; different AlarmSystem instances can mix and match different consequence combinations; easy to disable or swap consequences independently	Consequence classes will directly interact with GameEntity, breaks encapsulation of the responders, creates strong coupling if without an abstraction, while further abstraction is still unnecessary under current scope.

Justification: Chose **B**.

- **Alternative A** directly interact with the responders, every new consequence requires modification in `AlarmSystem`, which is **against Open-Closed Principle**.
- **Alternative C** centeralizes the consequence logic, but at the cost of breaking responder's encapsulation, which is **against the Information Hiding.
- **Alternative B** keeps every responder responsible for their own alarm behaviour, this adheres to the **Single Responsibility Principle**. `AlarmSystem` relies on the `AlarmSubscriber` Interface instead of interacting directly with responders, which adheres to **Dependency Inversion Principle**. Adding new responders does not require changes in `AlarmSystem`, this adheres to the **Open-Closed Principle**.

Trade off: Alarm logic is distributed rather than centralized, but for a system with two consequences, this distribution is still manageable.

We acknowledge that **Alternative C** offers better composability if multiple alarm systems with different consequence combinations were required, since the `Consequence`s could recombine independently from responders. But the REQ4 only required single alarm system with 2 consequences. Under this scope, the **Alternative B** can satisfy the requirement without introducing an extra abstraction layer for a possible requirement that does not even currently exist.

4.2 AlarmFloor and AlarmTrigger -- Factory Method

Requirement: A hidden floor tile that triggers an alarm when a worker stands on it, it must look the same as normal floors.

Approach:
The `AlarmTrigger` interface:

- has method `monitor(Location)` which specifies how a trigger is triggered.

The `AlarmFloor` class:

- extends `Ground` and `Implements AlarmTrigger`.
- being package-specific and takes an `AlarmSystem` in its contructor, the only way to obtain an `AlarmFloor` instance outside the `.alarm` package is through the factory method inside `AlarmSystem`, ensuring that it is always coupled to an `AlarmSystem`.
- The display character is passed in from constructor through factory method, set to `_` to imitate normal floors.

Reason:

- This approach intentionally coupled the `AlarmFloor` with the `AlarmSystem` to ensure safety, other classes cannot create any instance of `AlarmFloor` without an `AlarmSystem`, which adheres to **Information Hiding**.
- The system is still open to more various `AlarmTrigger`s under this approach, which adheres to the **Open-Closed Principle**.

4.3 FollowBehavior and Hunting

Requirement: When an alarm is triggered, all hostile entites will stop wandering and directly move to the worker who triggered the alarm.

Approach: in `onAlarmTriggered()`, replace the `WanderBehaviour` in hostile creature's tree map with `FollowBehaviour(target=targetWorker)`. The hostile creatures maintains an internal timer and swap back to `WanderBehaviour` when the alarm ends. The duration of huntdown is passed in through `onAlarmTriggered(target,alarmDuration)`'s alarmDuration parameter.

Reason: The tree map in REQ3 is exactly built for this. Behaviours are encapsulated into classes, and allows hotswap. `Behaviour` manage the specific `Action`s, Undead manage the `Behaviour` to take, this adheres to the **Single Responsibility Principle**. This also adheres to **Open-Closed Principle** where the change of creature's movement logic does not require a change in `Behaviour`, but this is the benefit from **Decision 3.1** instead of the current designing.

4.4 Lockdown of Doors

Requirement: When an alarm is triggered, all doors lock down for a set duration. `AccessCard` cannot override the lockdown.

Approach: The `Door` maintains a `lockdownDuration` counter. When the alarm triggers, `onAlarmTriggered()` sets this counter and forces the door locked. `tick()` decrements it each turn, the door automatically returns to normal when the counter reaches zero. During lockdown, `allowableActions()` hides the `UnlockDoorAction`, so `AccessCard` cannot bypass the lock.

Reason: `lockdown(duration)` is designed as a capability the `Door` possesses, the alarm merely invokes it. `Door` manages its own lockdown state and recovery internally, this adheres to the **Single Responsibility Principle**.

REQ4 SOLID Summary

The requirement specifically requires the system to be robust and scalable, to show that my implementation is so, I summarized the application of S.O.L.I.D. principles of this alarm system here.

Principle	Application in REQ4
SRP	`AlarmSystem` only dispatches notifications; `Door` manages its own lockdown; `Undead` manages its own behaviour swap; `AlarmFloor` only detects and triggers
OCP	New trigger types implement `AlarmTrigger`; new subscribers implement `AlarmSubscriber`, neither requires changes to `AlarmSystem` or existing responders
LSP	`AlarmFloor` substitutes `Ground` transparently; `FollowBehaviour` substitutes `Behaviour` without breaking the TreeMap loop in REQ3
ISP	`AlarmTrigger` has only `monitor(Location)`; `AlarmSubscriber` has only `onAlarmTriggered()`, no implementer is forced to depend on methods it does not use
DIP	`AlarmSystem` and `Door` both depend on `AlarmSubscriber`; `AlarmSystem` and `AlarmFloor` both depend on `AlarmTrigger`, both high and low level modules depend on the same abstraction, avoiding direct interactions